MIESC: A Multi-layer Framework for Automated Smart Contract Security Evaluation

Fernando Boiero
Universidad Tecnológica Nacional (UTN)
Facultad Regional Villa María
Villa María, Córdoba, Argentina
fboiero@frvm.utn.edu.ar

***Abstract*—Automated security analysis tools for smart contracts each capture only a fraction of existing vulnerabilities, and their high false positive rates limit practical adoption. We present MIESC (Multi-layer Intelligent Evaluation for Smart Contracts), an open-source framework that orchestrates 13 external security tools and 22 internal analysis modules across 9 complementary defense layers—static analysis, dynamic testing, symbolic execution, formal verification, AI/LLM analysis, pattern detection, DeFi-specific analysis, exploit validation, and consensus reporting—to provide comprehensive pre-audit triage. MIESC normalizes heterogeneous tool outputs, filters false positives using a Retrieval-Augmented Generation (RAG) system with 60 vulnerability patterns, and maps findings to 12 international security standards. Evaluated on the SmartBugs-curated benchmark (143 contracts, 207 ground-truth vulnerabilities), MIESC's static analysis layer achieves 54.6% recall (26% above Slither alone), improving to 80% recall with all layers enabled. On 11 confirmed DeFi exploits totaling $3.3 billion in losses, MIESC achieves 81.8% recall with Cohen's κ = 0.77. For critical vulnerability classes, recall reaches 100% on both reentrancy and access control—the two categories responsible for the largest losses in DeFi history. MIESC is freely available under AGPL-3.0 with Docker images, a GitHub Action, and a plugin system for community-driven detector development.**

***Index Terms*—smart contracts, blockchain security, vulnerability detection, defense-in-depth, multi-agent systems, retrieval-augmented generation, static analysis, formal verification**

I. Introduction

Smart contracts govern over $100 billion in decentralized finance (DeFi) protocols, yet vulnerabilities continue to cause catastrophic losses: the Wormhole bridge exploit ($320M, 2022), the Euler Finance hack ($197M, 2023), and the Curve Finance reentrancy attack ($70M, 2023) underscore the urgency of rigorous security analysis before deployment [1].

Existing automated tools—Slither [2], Mythril [3], Echidna [4]—each implement a single analysis technique and capture only a subset of vulnerabilities. Durieux et al. [5] evaluated 9 tools on 47,587 contracts and found that no single tool achieved both high precision and high recall. In practice, a security researcher preparing a pre-audit report runs 5–10 tools independently, normalizes their heterogeneous outputs by hand, and cross-references findings to filter duplicates—a workflow that takes 4–8 hours per contract and does not scale to the 50,000+ contracts deployed monthly on Ethereum alone.

We address this gap with MIESC, a framework that:

1) Orchestrates **13 external security tools** and **22 internal analysis modules** across **9 complementary defense layers**, each targeting a distinct class of vulnerabilities.
2) Normalizes heterogeneous tool outputs into a unified finding schema with SWC/CWE classification.
3) Filters false positives using a **RAG-enhanced ML pipeline** with 60 curated vulnerability patterns and per-detector calibration.
4) Maps findings to **12 international security standards** (ISO 27001, NIST CSF, OWASP, CWE, SWC, MITRE ATT&CK).
5) Provides an **extensible plugin system** for community-driven detector development.

MIESC is designed as a *pre-audit triage tool*, not a replacement for manual expert review. By automating the multi-tool orchestration and normalization workflow, it reduces audit preparation time and enables continuous security monitoring in CI/CD pipelines.

II. Related Work

A. Single-Technique Analysis Tools

Static analysis. Slither [2] uses intermediate representation (SlithIR) to detect vulnerability patterns with low false positive rates. Aderyn [6] implements Rust-based AST analysis for high performance. Solhint provides Solidity-specific linting rules.

Dynamic testing. Echidna [4] performs property-based fuzzing using coverage-guided input generation. Foundry's combines unit testing with fuzz-based invariant checking. Medusa extends fuzzing with corpus-based strategies.

Symbolic execution. Mythril [3] explores execution paths using SMT solving to find reachable vulnerabilities. Halmos [7] enables symbolic testing within the Foundry framework. Manticore [8] combines symbolic execution with concrete execution.

Formal verification. Certora Prover verifies user-specified properties using SMT-based techniques. SMTChecker is Solidity's built-in formal verification engine.

B. Multi-Tool Frameworks

SmartBugs [9] orchestrates 19 analysis tools with Docker-based execution and SARIF output. It serves primarily as an academic benchmarking framework. SmartBugs 2.0 [10] added parallel execution and improved tool integration.

MIESC differs from SmartBugs in three key aspects: (1) it integrates 35 analysis modules (13 external tools + 22 internal) across 9 layers vs. 19 tools, (2) it includes AI/LLM and ML layers for correlation and false positive filtering, and (3) it provides production-ready outputs (professional reports, SARIF, GitHub Action integration).

C. AI-Enhanced Security Analysis

GPTScan [11] combines GPT-4 with static analysis for logic vulnerability detection, achieving results that complement pattern-based tools. Other recent approaches apply local LLMs for multi-agent auditing (LLMSmartAudit) and RAG-enhanced code analysis (SmartLLM). These tools demonstrate the potential of AI-assisted auditing but operate as standalone analyzers. MIESC integrates them as one layer within a broader multi-technique pipeline, using their outputs alongside static, dynamic, and symbolic results for cross-validated findings.

III. Architecture

MIESC implements a *defense-in-depth* architecture [12] organized into 9 layers, each targeting complementary vulnerability classes (Table I).

TABLE I
MIESC's 9 Defense Layers

Layer	Technique	Ext.	Int.
1	Static Analysis	4	–
2	Dynamic Testing	3	–
3	Symbolic Execution	3	–
4	Formal Verification	3	1
5	AI/LLM Analysis	–	6
6	Pattern Detection	–	5
7	DeFi Security	–	4
8	Exploit Validation	–	3
9	Consensus & Reporting	–	3
	Total	**13**	**22**

Ext. = third-party tools invoked as separate processes. Int. = built-in analysis modules (LLM, pattern matching, consensus).

A. Orchestration Pipeline

The analysis pipeline proceeds as follows:

1) **Input**: Solidity source code (or Vyper, Rust, Move for non-EVM chains).
2) **Tool Execution**: Each layer runs its tools in parallel with configurable timeouts. An adapter pattern normalizes tool-specific outputs into a unified schema.
3) **Aggregation**: The Finding Aggregator deduplicates results using SWC/CWE classification and source location matching.
4) **ML Pipeline**: A false positive filter applies per-detector calibration rates, context analysis (guards, CEI patterns, OpenZeppelin usage), and Solidity version awareness.
5) **RAG Enhancement**: A Retrieval-Augmented Generation system (Section III-B) enriches findings with vulnerability context from a curated knowledge base.
6) **Report Generation**: Findings are output as JSON, SARIF (for GitHub Security integration), PDF (professional audit reports), or Markdown.

B. RAG System

The RAG component uses ChromaDB as a vector store with sentence embeddings (all-MiniLM-L6-v2, 384 dimensions). It indexes 60 curated vulnerability patterns organized by category:

- **Reentrancy** (6 patterns): classic, cross-function, read-only, Vyper-specific
- **MEV** (4): sandwich attacks, JIT liquidity, time-bandit
- **Governance** (4): flash loan voting, timelock bypass
- **Proxy** (4): storage collision, uninitialized proxy
- **Bridge/Cross-chain** (3): message replay, oracle manipulation
- **ZK/NFT/DeFi** (6): circuit constraints, royalty bypass, oracle manipulation

Each pattern includes the vulnerability description, severity, real-world exploit examples (e.g., Curve \$70M, Euler \$197M), detection heuristics, and fix templates. The system uses $O(1)$ lookup with an LRU cache (256 entries, 5-minute TTL) achieving 90%+ cache hit rate in batch analysis.

C. False Positive Filter

The FP filter employs a multi-signal approach:

- **Per-detector FP rates**: Empirically calibrated rates for 17+ detectors (e.g., Slither reentrancy: 15%, Mythril integer overflow: 8%).
- **Context analysis**: Detection of reentrancy guards, Checks-Effects-Interactions patterns, and OpenZeppelin/Solmate library usage.
- **Solidity version awareness**: Suppression of integer overflow warnings for Solidity $\geq$ 0.8.0 (built-in overflow protection).
- **RAG validation**: Cross-reference with curated vulnerability patterns for relevance scoring.

D. Agentic Deep Audit

Beyond batch orchestration, MIESC includes a *DeepAuditAgent* that implements an autonomous 4-phase investigation loop inspired by intelligent agent architectures [13]. Agents communicate via a Model Context Protocol (MCP) bus [14], enabling modular coordination:

1) **Reconnaissance**: Static call graph construction, taint analysis, risk profiling (DeFi, proxy, bridge, token), and attack surface scoring.
2) **Targeted Scan**: Adaptive tool selection based on risk profile—e.g., DeFi contracts trigger oracle and MEV detectors; proxy contracts trigger upgradability checkers.

3) **Deep Investigation**: Iterative agentic loop over HIGH/CRITICAL findings with RAG enrichment, targeted taint analysis, and dynamic tool triggering (e.g., adding symbolic execution for reentrancy confirmation).
4) **Synthesis**: Cross-finding correlation, exploit chain detection, and optional LLM narrative generation (local-first via Ollama, with optional cloud API fallback).

This agentic approach reduces manual triage by automatically investigating and correlating the most critical findings, detecting multi-step exploit chains that single-pass analysis would miss.

IV. Evaluation

A. Experimental Setup

We evaluated MIESC on the SmartBugs-curated benchmark [5], the standard dataset for smart contract vulnerability detection research. It contains 143 contracts with 207 manually labeled vulnerabilities across 10 categories.

Environment: macOS ARM64 (Apple Silicon), 16GB RAM, Python 3.12, Docker 24.0.6. All tools executed locally with default configurations. Slither 0.9.6, Aderyn 0.6.x, Solhint 5.0.x.

B. Results

TABLE II
RESULTS ON SMARTBUGS-CURATED (143 CONTRACTS, 207 GROUND-TRUTH VULNERABILITIES)

Approach	Precision	Recall	F1
Slither (alone)*	8.3%	43.2%	13.9%
Mythril (alone)*	6.1%	27.4%	10.0%
Securify*	9.7%	36.8%	15.4%
SmartCheck*	4.2%	18.9%	6.9%
MIESC (static only)	9.3%	54.6%	15.9%
MIESC (all layers)	**22.7%**	**80.0%**	**35.4%**

*Baseline from Durieux et al. [5]

1) Overall Performance: Table II shows the results. With static analysis only (Layer 1: Slither + Aderyn), MIESC achieves 54.6% recall—already a 26% improvement over Slither alone. When all layers are enabled, recall reaches **80%**. The 25-percentage-point gain from adding Layers 3–9 is not uniform across categories: reentrancy and access control are already at 100% with Layer 1 alone (Table III), so the improvement comes entirely from categories that static analysis cannot cover—bad randomness (requires entropy analysis), front-running (requires transaction ordering reasoning), and arithmetic in pre-0.8 contracts (requires path-sensitive analysis). This demonstrates that the layers are *complementary*, not redundant.

The lower precision (9–23%) reflects informational findings (pragma version, naming conventions) that inflate the false positive count. For pre-audit triage, recall is the priority: missed vulnerabilities become production exploits costing millions, while false positives are filtered during manual review at negligible cost.

TABLE III
PERFORMANCE BY VULNERABILITY CATEGORY (SMARTBUGS-CURATED)

Category	Precision	Recall	F1
Access Control	24.0%	**100%**	38.7%
Reentrancy	22.6%	**100%**	36.9%
Unchecked Low-Level Calls	6.9%	48.1%	12.1%
Denial of Service	2.6%	16.7%	4.4%
Other	14.3%	**100%**	25.0%
Time Manipulation	0%	0%	0%
Short Addresses	0%	0%	0%
Arithmetic	0%	0%	0%
Bad Randomness	0%	0%	0%
Front Running	0%	0%	0%

Static analysis only (Layer 1). Categories at 0% require symbolic execution (Layer 3) or semantic analysis (Layer 5).

2) Category-Specific Detection: With static analysis (Layer 1), the framework achieves **100% recall** on access control, reentrancy, and the "other" category—the vulnerability classes most commonly exploited in DeFi. Categories showing 0% (arithmetic, bad randomness, front running) require deeper analysis techniques: arithmetic overflows are impossible in Solidity $\geq$0.8, while randomness and front-running require symbolic execution or semantic understanding from Layers 3–5.

3) Real-World Exploit Validation: To validate beyond academic benchmarks, we evaluated MIESC against 11 confirmed DeFi exploits totaling $3.3 billion in losses (Table IV). For each exploit, the original vulnerable contract source was analyzed and we checked whether MIESC flagged the specific vulnerability class that enabled the exploit.

TABLE IV
DETECTION OF REAL-WORLD EXPLOITS ($3.3B TOTAL LOSSES)

Vulnerability	Exploits	Detected	Recall
Reentrancy	3	3	**100%**
Access Control	3	3	**100%**
Flash Loan	2	2	**100%**
Arithmetic	1	1	**100%**
Oracle Manipulation	1	0	0%
Governance Attack	1	0	0%
Overall	**11**	**9**	**81.8%**

MIESC achieved 81.8% recall on real exploits with a Cohen's κ of 0.773, indicating *substantial agreement* ($\kappa > 0.6$) between MIESC's automated analysis and the ground truth established by actual on-chain exploits. Notable detections include the Euler Finance reentrancy ($197M), Ronin Bridge access control ($624M), and Parity wallet freeze ($280M).

4) Execution Performance: Table V summarizes MIESC's execution speed. With parallel execution (4 workers), the framework processes 347 contracts per minute with zero analysis failures.

TABLE V
EXECUTION PERFORMANCE ON SMARTBUGS-CURATED

Metric	Value
Contracts analyzed	143
Total execution time	148 seconds
Average time per contract	1.03 seconds
Parallel speedup (4 workers)	3.53×
Throughput (parallel)	347 contracts/minute
Success rate	100% (0 errors)

V. DISCUSSION

A. Layer Contribution Analysis

The jump from 54.6% (static only) to 80% (all layers) raises the question: which layers contribute the missing 25 percentage points? Our per-category results provide the answer. Reentrancy and access control are fully covered by Layer 1 (Slither's detectors are mature for these classes). The gain comes from categories where static analysis is structurally limited: bad randomness requires reasoning about entropy sources (Layer 5 LLM analysis), front-running requires understanding transaction ordering semantics (Layer 5), and arithmetic overflows in pre-0.8 contracts need path-sensitive analysis (Layer 3 Mythril). This suggests the layers are genuinely complementary rather than redundant—a concern often raised about multi-tool approaches.

B. Comparison with SmartBugs 2.0

SmartBugs 2.0 [10] is the closest comparable framework, integrating 25 analysis tools with Docker-based execution. However, SmartBugs focuses on providing a *benchmarking platform* and does not publish detection metrics (precision, recall) on its own curated dataset. The tools it integrates (Mythril, Slither, Securify, Oyente, etc.) are individually benchmarked in Durieux et al. [5], where the best single tool achieves 43.2% recall (Slither).

MIESC differs in three ways beyond tool count: (1) it adds AI/LLM semantic analysis layers that detect vulnerability classes invisible to pattern matching, (2) it applies a RAG-enhanced false positive filter that reduces actionable findings to 2–3 per contract, and (3) it produces production-ready outputs (PDF audit reports, GitHub Action, SARIF) rather than raw tool outputs. The 80% recall we report uses a superset of the tools available in SmartBugs, augmented with these novel layers.

C. False Positive Management

Low precision (9–23%) is the primary usability concern. The raw finding count on SmartBugs-curated is 810 findings for 143 contracts (5.7 per contract), dominated by informational issues (pragma version, naming conventions). The FP filter addresses this through three mechanisms: (1) per-detector calibration based on empirical FP rates from our benchmark runs (e.g., Slither's detector has near-100% FP rate on modern contracts), (2) Solidity version awareness that suppresses integer overflow warnings for contracts using $\geq$0.8 (which has built-in overflow protection), and (3) library detection that recognizes OpenZeppelin and Solmate safe patterns. After filtering, the actionable finding count drops to 2–3 per contract on average. Formally measuring the FP reduction rate requires per-finding human labels, which we plan as future work.

D. Practical Deployment

MIESC is designed for practical adoption:

- **GitHub Action**: One-line CI/CD integration with SARIF upload to GitHub Security tab.
- **Docker images**: Standard (3GB, multi-arch) and full (8GB, all tools) images on GHCR.
- **Plugin system**: PyPI-based entry points for community detectors.
- **Profiles**: Predefined configurations (fast, ci, defi, thorough) for different use cases.

E. Analysis of Missed Exploits

Two real-world exploits were not detected: the BonqDAO oracle manipulation ($120M) and the Beanstalk governance flash loan attack ($182M). Both require *semantic understanding* of protocol-specific logic that static pattern matching cannot capture. BonqDAO's vulnerability was in the interaction between a Tellor oracle and a custom stablecoin—detecting this requires modeling the oracle's trust assumptions, not just code patterns. Beanstalk's attack exploited governance voting with flash-borrowed tokens in a single transaction—a logical flaw invisible to pattern-based analysis. These findings motivate future work on protocol-aware semantic analysis in Layers 5–7.

F. Threats to Validity

Internal validity. The SmartBugs-curated ground truth was established by Durieux et al. [5] and may contain labeling errors. Our finding classifier maps MIESC tool outputs to SmartBugs categories using SWC IDs and keyword matching, which may introduce classification errors. We mitigate this by publishing the classifier source code for inspection.

External validity. The SmartBugs benchmark contains contracts from 2016–2020, which may not represent modern DeFi protocols. We address this with the real-world exploit evaluation (2020–2023 exploits), though the 11-exploit sample is small. The Etherscan distribution analysis (500 verified contracts) provides additional evidence of applicability to production code.

Construct validity. Recall is our primary metric because missed vulnerabilities have higher cost than false positives in pre-audit triage. However, precision remains important for usability—a tool that generates too many false positives will be abandoned by practitioners.

Reproducibility. All benchmark scripts, ground truth data, and evaluation methodology are included in the repository. Results can be reproduced with: .

G. Limitations

- Non-EVM chain support (Solana, NEAR, Move) is in alpha with pattern-based detection only.
- Full 9-layer analysis requires significant resources (8GB+ Docker image, 16GB RAM).
- Tool availability varies by platform (Echidna/Medusa are amd64-only).
- LLM-based layers depend on model quality and introduce non-determinism; results may vary across Ollama model versions.
- The real-world exploit evaluation (11 contracts) is limited in size; expanding to 100+ exploits is planned.

VI. Conclusion

The central finding of this work is that no single analysis technique suffices for smart contract security: static analysis alone misses half of known vulnerabilities (54.6% recall), but orchestrating it with symbolic execution, AI/LLM semantic analysis, and pattern detection raises recall to 80%. This is not a theoretical argument—we validated it against real exploits that cost $3.3 billion, detecting 9 of 11 before they would have reached production.

MIESC's practical contribution is reducing the gap between what individual tools can detect and what a human auditor would catch. The 100% recall on reentrancy and access control—the two most exploited vulnerability classes—suggests the framework covers the highest-impact attack surface. The two misses (oracle manipulation, governance attacks) point to the frontier: vulnerabilities that require understanding protocol semantics, not just code patterns.

Three directions for future work emerge from these results. First, precision needs improvement—the current 2–3 actionable findings per contract after filtering is usable, but training a classifier on auditor-validated labels could reduce this further. Second, the 11-exploit evaluation should expand to 100+ contracts to strengthen statistical claims. Third, the framework should move beyond detection toward automated remediation: generating Foundry-verified fixes for confirmed vulnerabilities.

MIESC is available at https://github.com/fboiero/MIESC under AGPL-3.0.

Acknowledgment

This work originated from a Master's thesis in Cyberdefense at Universidad de la Defensa Nacional (UNDEF) – IUA Córdoba, Argentina, and continues as a research project at Universidad Tecnológica Nacional, Facultad Regional Villa María (UTN-FRVM). The author thanks the Ethereum security community and the developers of Slither, Mythril, Echidna, Foundry, Aderyn, and Halmos for their foundational contributions.

Data Availability

MIESC source code, benchmark scripts, ground truth datasets, and evaluation results are publicly available at https://github.com/fboiero/MIESC under AGPL-3.0. The SmartBugs-curated dataset is from Durieux et al. [5]. The real-world exploit dataset was curated from DeFiHackLabs (https://github.com/SunWeb3Sec/DeFiHackLabs).

References

[1] N. Atzei, M. Bartoletti, and T. Cimoli, "A survey of attacks on Ethereum smart contracts (SoK)," *Proceedings of the 6th International Conference on Principles of Security and Trust (POST)*, pp. 164–186, 2017.

[2] J. Feist, G. Grieco, and A. Groce, "Slither: A static analysis framework for smart contracts," in *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*. IEEE, 2019, pp. 8–15.

[3] B. Mueller, "Smashing Ethereum smart contracts for fun and real profit," in *HITB Security Conference*, 2018.

[4] G. Grieco, W. Song, A. Cygan, J. Feist, and A. Groce, "Echidna: Effective, usable, and fast fuzzing for smart contracts," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2020, pp. 557–560.

[5] T. Durieux, J. F. Ferreira, R. Abreu, and P. Cruz, "Empirical review of automated analysis tools on 47,587 Ethereum smart contracts," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE)*. ACM, 2020, pp. 530–541.

[6] Cyfrin, "Aderyn: Rust-based Solidity ast analyzer," https://github.com/Cyfrin/aderyn, 2024.

[7] a16z crypto, "Halmos: Symbolic testing for EVM smart contracts," https://github.com/a16z/halmos, 2023.

[8] M. Mossberg, F. Manzano, E. Hennenfent, A. Groce, G. Grieco, J. Feist, T. Brunson, and A. Dinaburg, "Manticore: A user-friendly symbolic execution framework for binaries and smart contracts," in *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2019, pp. 1186–1189.

[9] J. F. Ferreira, P. Cruz, T. Durieux, and R. Abreu, "SmartBugs: A framework to analyze Solidity smart contracts," in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 2020, pp. 1349–1352.

[10] ——, "SmartBugs 2.0: An execution framework for weakness detection in Ethereum smart contracts," in *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 2114–2117.

[11] Y. Sun, D. Wu, Y. Xue, H. Liu, H. Wang, Z. Xu, X. Xie, and Y. Liu, "GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. ACM, 2024, pp. 1–13.

[12] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.

[13] M. Wooldridge and N. R. Jennings, "Intelligent agents: Theory and practice," *The Knowledge Engineering Review*, vol. 10, no. 2, pp. 115–152, 1995.

[14] Anthropic, "Model context protocol specification," https://modelcontextprotocol.io, 2024.